COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	8/18/2026	**DATE SUBMITTED:**	8/20/2026
NAME:	Dimaguiba, Lorenzo James M.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HANDS-ON ACTIVITY NO. 6.1 **SEARCHING TECHNIQUES**

1. PROCEDURE OUTPUT

CODE

main.cpp | searchingAlgo.h | node.h

Search Trees > main.cpp > ...

```
1   #include<iostream>
2   #include <cstdlib>
3   #include <time.h>
4   #include "node.h"
5   #include "searchingAlgo.h"
6
7   const int max_size = 50;
8
9   int main(){
10
11      //generate
12      int dataset[max_size];
13      srand(time(0));
14      for (int i=0; i < max_size; i++){
15          dataset[i] = rand();
16      }
17
```

OUTPUT

```
3832 21034 29400 7725 20349 20303 19334 32522 23155 23305 712 10553 20046 393 32087 4763 7477 12520 25403 11308 30984
6 24297 11313 17779 18829 5136 7358 17412 23669 12743 14738 8158 29983 6494 19273
```

Table 6-1. Data Generated and Observations

CODE

Node header:

main.cpp searchingAlgo.h node.h

Search Trees > C node.h > ...

```
1   #ifndef NODE_H
2   #define NODE_H
3
4   template<typename T>
5   class Node{
6       public:
7       T data;
8       Node<T>* next;
9   };
10
11  template <typename T>
12  Node<T>* new_node(T newData){
13      Node<T>* newNode = new Node<T>;
14      newNode->data = newData;
15      newNode->next = nullptr;
16      return newNode;
17  }
18
19
20
21  #endif
```

Function headers:

main.cpp | searchingAlgo.h | node.h

Search Trees > searchingAlgo.h > binarySearchAR<T>(int, T, T [])

```
#ifndef SEARCHINGALGO_H
#define SEARCHINGALGO_H
#include "node.h"
/*
N -> Boundary of the list
Item -> Searching number
Data -> Linear array
Step 1: I := 0
Step 2: Repeat while I <= n
If (item = data[i])
Print "Searching is successful"
Exit
Else
Print "Searching is Unsuccessful"
Exit*/

template <typename T>
void linearSearchArr(int arrSize, T dataFind, T list[]){
//Step 1: I := 0
    int i = 0;
//Step 2: Repeat while I <= n
//_____________________________________
    while(i < arrSize){
        //If (item = data[i])
        if (dataFind == list[i]){
            //Print "Searching is successful"
            std::cout<<"Searching is successful!\n";
            return;
        }
        i++;
    }
    //Print "Searching is Unsuccessful"
    std::cout<<"Searching is Unsuccessful"<<std::endl;
//_____________________________________

    //alt
/*
for (int i = 0; i < arrSize; i++){
    if (dataFind == list[i]){
            //Print "Searching is successful"
            std::cout<<"Searching is successful!\n";
            return;
        }
    //Print "Searching is Unsuccessful"
    std::cout<<"Searching is Unsuccessful"<<std::endl; */
}

template <typename T>
void linearSearchLL(T dataFind, Node<T>* head){
    while(head != nullptr){
        if (head->data == dataFind){
            std::cout<<"Searching Successful!"<<std::endl;
            return;
        }
        head = head->next;
    }
    std::cout<<"Search Unsuccessfull!"<<std::endl;
}
```

Driver:

main.cpp | searchingAlgo.h | node.h

Search Trees > main.cpp > ...

```cpp
#include<iostream>
#include <cstdlib>
#include <time.h>
#include "node.h"
#include "searchingAlgo.h"

const int max_size = 50;

int main(){

    //generate
    int dataset[max_size];
    srand(time(0));
    for (int i=0; i < max_size; i++){
        dataset[i] = rand();
    }

    for (int i = 0; i < max_size; i++){
        std::cout<<dataset[i]<<" ";
    }
    std::cout<<std::endl;
    int userInput;
    std::cout<<"What number to search?";
    std::cin >> userInput;
    linearSearchArr(max_size, userInput, dataset);

    //create linked list
    Node<char> * name1 = new_node('J');
    Node<char> * name2 = new_node('A');
    Node<char> * name3 = new_node('M');
    Node<char> * name4 = new_node('E');
    Node<char> * name5 = new_node('S');

    name1->next = name2;
    name2->next = name3;
    name3->next = name4;
    name4->next = name5;

    linearSearchLL('M', name1);
    linearSearchLL('r', name1);

```

OUTPUT

```
3832 21034 29400 7725 20349 26
6 24297 11313 17779 18829 5136
What number to search?5136
Searching is successful!
```

Table 6-2a. Linear Search for Arrays

CODE

Function headers:

```
47
48  template <typename T>
49  void linearSearchLL(T dataFind, Node<T>* head){
50      while(head != nullptr){
51          if (head->data == dataFind){
52              std::cout<<"Searching Successful!"<<std::endl;
53              return;
54          }
55          head = head->next;
56      }
57      std::cout<<"Search Unsuccessfull!"<<std::endl;
58  }
59
```

Driver:

```
27      //create linked list
28      Node<char> * name1 = new_node('J');
29      Node<char> * name2 = new_node('A');
30      Node<char> * name3 = new_node('M');
31      Node<char> * name4 = new_node('E');
32      Node<char> * name5 = new_node('S');
33
34      name1->next = name2;
35      name2->next = name3;
36      name3->next = name4;
37      name4->next = name5;
38
39      linearSearchLL('M', name1);
40      linearSearchLL('r', name1);
41
```

OUTPUT

```
3832 21034 29400 7725 20349 26
6 24297 11313 17779 18829 5136
What number to search?5136
Searching is successful!
Searching Successful!
Search Unsuccessfull!
```

Table 6-2b. Linear Search for Linked List

CODE

Header:

```
127   //for array binary search
128   template <typename T>
129   void binarySearchAR(int arrSize, T dataFind, T list[]){
130   //define low, up, middle
131   int low = 0, up = arrSize -1, middle;
132
133   //loop while low is less than or equal to high
134   while(low <= up){
135       //compute for middle
136       middle = ((low + up)/2);
137       //if data found is middle -> data found -> return
138       if (dataFind == list[middle]){
139           std::cout<<"The search element is found!\n";
140           return;
141       }
142       else if (dataFind < list[middle]){
143           up = middle -1;
144       }
145
146       else{
147           low = middle + 1;
148       }
149   }
150
151   //search not found
152   std::cout<<"Search element not found"<<std::endl;
153
154   }
```

Driver:

```
42      //binary driver code
43      Node<int> * num1 = new_node(1);
44      Node<int> * num2 = new_node(2);
45      Node<int> * num3 = new_node(3);
46      Node<int> * num4 = new_node(4);
47      Node<int> * num5 = new_node(5);
48
49      num1->next = num2;
50      num2->next = num3;
51      num3->next = num4;
52      num4->next = num5;
53
54      binarySearchLL(num1, 4);
55      binarySearchLL(num1, 0);
56      int test[8] = {1,2,3,4,5,6,7,8}; //must be ordered
57      binarySearchAR(max_size, 12, test);
58      binarySearchAR(max_size, 1, test);
59
60      return 0;
61  }
```

OUTPUT

```
Search element not found
The search element is found!
```

Note: list must be ordered to work properly

Table 6-3a. Binary Search for Arrays

Observation/Analysis:

There are two types of searching algorithms or techniques. Each with their own use and function. Those types being Linear and Binary Search. Linear search is more suited for unsorted data, or data that is unordered. Linear search, if in an array, through a loop, compares every element with a specific value or in other words, a key until it finds a match and returns a result if found. The search will be a failure or unsuccessful otherwise. If done through a linked list, it uses traversal as a way to compare each node to the key. Binary Search is used for ordered data (usually in ascending order) in which it halves the time to search and the area of search by halving the list or array by finding the middle node till it finds a match. If not, it ends the same way with a search fail or unsuccessful search.

CODE

Middle Node:

```cpp
/*1. Traverse the singly linked list using two pointers.
2. Move one pointer by one step ahead and the other pointer by two steps.
3. When the fast pointer reaches the end of the singly linked list, the slow pointer will
reach the middle of the singly linked list.
4. Return slow pointer address*/
template <typename T>
Node<T>* getMiddle(Node<T>* start, Node<T>* last){

    Node<T>* slow = start;
    Node<T>* fast = start;

    while (fast != last && fast->next != last){

            //move slow one step
            slow = slow->next;

            //move fast two steps
            fast = fast->next->next;
    }

    return slow;
}
```

Header:

```cpp
template <typename T>
void binarySearchLL(Node<T>* start, T dataFind){
   //1. Start node is set to head of the list and the last node is set to NULL.
    Node<T>* last = nullptr;


    while(start != last){

    //2. Middle element is calculated using the two pointers approach discussed above.
    Node<T>* middle = getMiddle(start, last); //program will not end

    //3. If the middle element is same as the key to be searched, we return it.
    if(middle->data == dataFind){
        std::cout<<"Search Successful!"<<std::endl;
        return;
    }

    //4. Else if middle element is greater than the key to be searched, we have to search is the left side of the singly linked list.
    else if(middle->data > dataFind){
        last = middle;
    }

    //5. Else if middle element is less than the key to be searched, we have to search is the right side of the singly linked list. S
    else{
        start = middle->next;
    }
    //6. If the key is found or the entire linked list gets traversed, we break the loop
}

std::cout<<"Search Unsuccessful!"<<std::endl;
return;
}
```

Driver:

```
42      //binary driver code
43      Node<int> * num1 = new_node(1);
44      Node<int> * num2 = new_node(2);
45      Node<int> * num3 = new_node(3);
46      Node<int> * num4 = new_node(4);
47      Node<int> * num5 = new_node(5);
48
49      num1->next = num2;
50      num2->next = num3;
51      num3->next = num4;
52      num4->next = num5;
53
54      binarySearchLL(num1, 4);
55      binarySearchLL(num1, 0);
56      int test[8] = {1,2,3,4,5,6,7,8}; //must be ordered
57      binarySearchAR(max_size, 12, test);
58      binarySearchAR(max_size, 1, test);
59
60      return 0;
61  }
```

OUTPUT

```
Search Successful!
Search Unsuccessful!
```

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

UNIVERSAL NODE HEADER:

Search Trees > Supplementary > C node.h > ...

```
1   #ifndef NODE_H
2   #define NODE_H
3
4   template<typename T>
5   class Node{
6       public:
7       T data;
8       Node<T>* next;
9   };
10
11  template <typename T>
12  Node<T>* new_node(T newData){
13      Node<T>* newNode = new Node<T>;
14      newNode->data = newData;
15      newNode->next = nullptr;
16      return newNode;
17  }
18
19
20
21  #endif
```

PROBLEM 1:

CODE

MAIN:

Search Trees > Supplementary > Dimaguiba_Problem1.cpp > main()

```cpp
#include <iostream>
#include "node.h"
#include"searchAlgo.h"
//set
const int max_capacity = 10;

int main(){
    //PROBLEM 1
    int list[max_capacity] = {5, 18, 2, 19, 18, 0, 8, 14, 19, 14};

    //array
    std::cout<<"Current list: ";
    for (int i = 0; i < max_capacity; i++){
        std::cout<<list[i]<<" ";
    }
    std::cout<<std::endl;
    int userInput;
    std::cout<<"Search for what number?: ";
    std::cin>>userInput;
    linearSearchArr(max_capacity, userInput, list);

    //linked list
    //create linked list
    Node<int> * num1 = new_node(5);
    Node<int> * num2 = new_node(18);
    Node<int> * num3 = new_node(2);
    Node<int> * num4 = new_node(19);
    Node<int> * num5 = new_node(18);
    Node<int> * num6 = new_node(0);
    Node<int> * num7 = new_node(8);
    Node<int> * num8 = new_node(14);
    Node<int> * num9 = new_node(19);
    Node<int> * num10 = new_node(14);

```

```
35      num1->next = num2;
36      num2->next = num3;
37      num3->next = num4;
38      num4->next = num5;
39      num5->next = num6;
40      num6->next = num7;
41      num7->next = num8;
42      num8->next = num9;
43      num9->next = num10;
44
45      linearSearchLL(userInput, num1);
46      return 0;
47  }
48
```

HEADER:

Search Trees > Supplementary > searchAlgo orig.h > linearSearchLL<T>(T, Node<T>*)

```
1   #ifndef SEARCHINGALGO_H
2   #define SEARCHINGALGO_H
3   #include "node.h"
4
5   //array
6   template <typename T>
7   void linearSearchArr(int arrSize, T findData, T list[]){
8       int comparisons = 0;
9       for (int i = 0; i < arrSize; i++){
10          comparisons = i;
11          if (findData == list[i]){
12              std::cout<<"Data found! Number of Comparisons: "<<++comparisons<<std::endl;
13              return;
14          }
15      }
16      std::cout<<"Unsuccessful Search!"<<std::endl;
17  }
18
19  //linked list
20  template <typename T>
21  void linearSearchLL(T findData, Node<T>* head){
22      int comparisons = 0;
23      while(head != nullptr){
24          ++comparisons;
25          if(head->data == findData){
26              std::cout<<"Data found! Number of Comparisons: "<<comparisons<<std::endl;
27              return;
28          }
29          head = head->next;
30      }
31      std::cout<<"Unsuccessful Search!"<<std::endl;
32  }
33
34  //display current list
35  template <typename T>
36  void viewAll(Node<T>* head){
37      if(head == nullptr){
38          std::cout<<"List Empty!"<<std::endl;
39          return;
40      }
41      std::cout<<"Current Linked List: ";
42      while(head != nullptr){
43          std::cout<<head->data<<" ";
44          head = head->next;
45      }
46      std::cout<<std::endl;
47  }
48
49  #endif
```

OUTPUT

```
Current list: 5 18 2 19 18 0 8 14 19 14
Search for what number?: 18
Data found! Number of Comparisons: 2
Current Linked List: 5 18 2 19 18 0 8 14 19 14
Data found! Number of Comparisons: 2
PS C:\Users\shosh\OneDrive\Documents\VSCODES>
```

PROBLEM 2:

CODE

Driver:

Search Trees > Supplementary > Dimaguiba_Problem1-2.cpp > main()

```cpp
#include <iostream>
#include "node.h"
#include"searchAlgo.h"
//set
const int max_capacity = 10;

int main(){
    //PROBLEM 1 - 2
    int list[max_capacity] = {5, 18, 2, 19, 18, 0, 8, 14, 19, 14};

    //array
    std::cout<<"Current list: ";
    for (int i = 0; i < max_capacity; i++){
        std::cout<<list[i]<<" ";
    }
    std::cout<<std::endl;
    int userInput;
    std::cout<<"Search for what number?: ";
    std::cin>>userInput;
    linearSearchArr(max_capacity, userInput, list);
    linearInstance(max_capacity, userInput, list);

    //linked list
    //create linked list
    Node<int> * num1 = new_node(5);
    Node<int> * num2 = new_node(18);
    Node<int> * num3 = new_node(2);
    Node<int> * num4 = new_node(19);
    Node<int> * num5 = new_node(18);
    Node<int> * num6 = new_node(0);
    Node<int> * num7 = new_node(8);
    Node<int> * num8 = new_node(14);
    Node<int> * num9 = new_node(19);
```

```
34      Node<int> * num10 = new_node(14);
35
36      num1->next = num2;
37      num2->next = num3;
38      num3->next = num4;
39      num4->next = num5;
40      num5->next = num6;
41      num6->next = num7;
42      num7->next = num8;
43      num8->next = num9;
44      num9->next = num10;
45
46      viewAll(num1); // views entire list
47      linearSearchLL(userInput, num1);
48      linearInstanceLL(userInput, num1);
49
50      return 0;
51  }
52
```

NEW HEADER COMMANDS (STILL SAME FILE):

```
19  template <typename T>
20  void linearInstance(int arrSize, T findData, T list[]){
21      int instances = 0;
22          for (int i = 0; i < arrSize; i++){
23              if(list[i] == findData){
24                  instances++;
25              }
26          }
27          if(instances == 0){
28              std::cout<<"Data not found!"<<std::endl;
29              return;
30          }
31          std::cout<<"Numer of instances: "<<instances<<std::endl;
32  }
```

```
49  template <typename T>
50  void linearInstanceLL(T findData, Node<T>* head){
51      int instances = 0;
52      if(head == nullptr){
53          std::cout<<"Nothing to search!"<<std::endl;
54          return;
55      }
56      while(head != nullptr){
57
58          if (head->data == findData){
59              instances++;
60          }
61          head = head->next;
62      }
63      if (instances == 0){
64          std::cout<<"Data not found!"<<std::endl;
65          return;
66      }
67      std::cout<<"Numver of instances: "<<instances<<std::endl;
68  }
```

OUTPUT

SUCCESS:

```
Current list: 5 18 2 19 18 0 8 14 19 14
Search for what number?: 19
Data found! Number of Comparisons: 4
Numer of instances: 2
Current Linked List: 5 18 2 19 18 0 8 14 19 14
Data found! Number of Comparisons: 4
Numver of instances: 2
PS C:\Users\shosh\OneDrive\Documents\VSCODES>
```

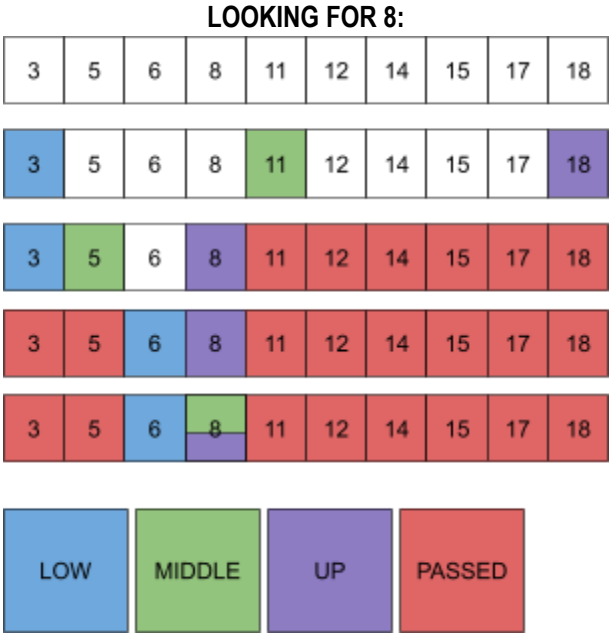
FAIL:

```
Current list: 5 18 2 19 18 0 8 14 19 14
Search for what number?: 67
Unsuccessful Search!
Data not found!
Current Linked List: 5 18 2 19 18 0 8 14 19 14
Unsuccessful Search!
Data not found!
PS C:\Users\shosh\OneDrive\Documents\VSCODES>
```

PROBLEM 3:

CODE

Drawing:



Header:

```cpp
//BINARY SEARCH Array
template <typename T>
void binarySearchArr(int arrSize, T findData, T list[]){
    int low = 0 , up = arrSize-1, middle;
    while(low <= up){
        middle = ((low + up)/2);
        std::cout<<"Current Mid: "<<list[middle]<<std::endl; //show current mid
        if (findData == list[middle]){
            std::cout<<"Search Complete! Found at index: "<<middle<<std::endl;
            return;
        }

        else if (findData < list[middle]){
            up = middle - 1;
        }

        else{
            low = middle + 1;
        }

    }

    std::cout<<"Search element not found"<<std::endl;
}
```

(Optional: works with linked list too):

```cpp
//Linked list version:
//get middle
template <typename T>
Node<T>* getMiddle(Node<T>* start, Node<T>* last){
    Node<T>* slow = start;
    Node<T>* fast = start;

    while(fast != last && fast->next != last){
        slow = slow->next;
        fast = fast->next->next;
    }

    return slow;
}

```

```cpp
template <typename T>
void binarySearchLL(Node<T>*start, T findData){
    Node<T>* last = nullptr;

    while(start != last){

        Node<T>*middle = getMiddle(start, last);

        if(middle->data == findData){
            std::cout<<"Search Successful!"<<std::endl;
            return;
        }

        else if(middle->data < findData){
            start = middle->next;
        }

        else{
            last = middle;
        }
    }

    std::cout<<"Search Unsuccessful!"<<std::endl;
}
#endif
```

Driver:

```cpp
#include <iostream>
#include "node.h"
#include"searchAlgo.h"
//set
const int max_capacity = 10;

int main(){
    //PROBLEM 3
    int list[max_capacity] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
    std::cout<<"Current list: ";
    for (int i = 0; i < max_capacity; i++){
        std::cout<<list[i]<<" ";
    }
    std::cout<<std::endl;

    int userInput;
    std::cout<<"Search for what number?: ";
    std::cin>>userInput;

    binarySearchArr(max_capacity, userInput, list);

```

```
22      //Linked list:
23      Node<int> * num1 = new_node(3);
24      Node<int> * num2 = new_node(5);
25      Node<int> * num3 = new_node(6);
26      Node<int> * num4 = new_node(8);
27      Node<int> * num5 = new_node(11);
28      Node<int> * num6 = new_node(12);
29      Node<int> * num7 = new_node(14);
30      Node<int> * num8 = new_node(15);
31      Node<int> * num9 = new_node(17);
32      Node<int> * num10 = new_node(18);
33
34      num1->next = num2;
35      num2->next = num3;
36      num3->next = num4;
37      num4->next = num5;
38      num5->next = num6;
39      num6->next = num7;
40      num7->next = num8;
41      num8->next = num9;
42      num9->next = num10;
43
44      binarySearchLL(num1, 18);
45
46      return 0;
47  }
48
```

OUTPUT

```
Current list: 3 5 6 8 11 12 14 15 17 18
Search for what number?: 8
Current Mid: 11
Current Mid: 5
Current Mid: 6
Current Mid: 8
Search Complete! Found at index: 3
Search Successful!
```

PROBLEM 4:

CODE

Header:

```
110   //recursive
111
112   template <typename T>
113   void RecursiveSearchArr(int arrSize, int low, int up, T findData, T list[]){
114       int middle;
115       if(up >= low){
116           middle = ((low + up)/2);
117           if (findData == list[middle]){
118               std::cout<<"Search Complete! Found at index: "<<middle<<std::endl;
119               return;
120           }
121
122           else if (findData < list[middle]){
123               return RecursiveSearchArr(arrSize, low, middle-1, findData, list);
124           }
125
126           else{
127               return RecursiveSearchArr(arrSize, middle+1, up, findData, list);
128           }
129
130       }
131
132       std::cout<<"Search element not found"<<std::endl;
133   }
134
```

Driver:

```
    //recursive:
    RecursiveSearchArr(max_capacity, 0, 9, 8, list);
    return 0;
}
```

OUTPUT

```
Current list: 3 5 6 8 11 12 14 15 17 18
Search for what number?: 8
Current Mid: 11
Current Mid: 5
Current Mid: 6
Current Mid: 8
Search Complete! Found at index: 3
Search Successful!
Search Complete! Found at index: 3
PS C:\Users\shosh\OneDrive\Documents\VSCODES>
```

Observation/Analysis:
Everything works the same as before, the Linear search, the Binary Search, all still works the same, with the added benefit of tracking the current index of the element if found in the list. It also counts the number of comparisons and instances there are of an element inside of a list. What's further explored is the graphical representation of how binary searching works providing a clearer understanding of it. The way recursion works is that a function calls itself multiple times until a certain condition is met. This can be observed on problem 4, in that recursion is utilized with similar properties as its array counterpart being that the low and high changes where the middle part is. It all ends the same when the condition is met or not.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.